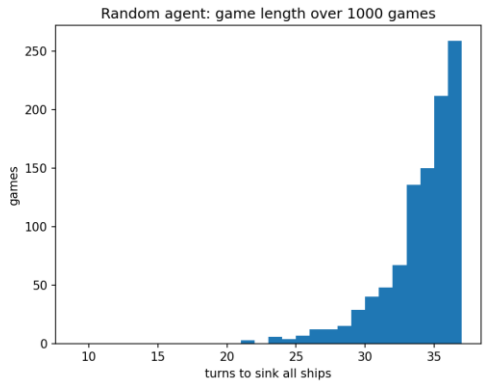
Lab 4: Representation Learning via Modern RL

Oliver Grudzinski

[n] marks a sentence that answers question n.

The `Battleship` class keeps two 6 × 6 boards: `true_state`, the hidden fleet (1 = ship), and `state`, what the agent sees (1 = hit, -1 = miss, 0 = not shot). `position_maker` picks a ship's first cell so the ship fits on the board, `check_empty` checks that it wouldn't overlap another ship, and `place_ship` resamples the position and a 50/50 orientation until it fits. `make_board` places all three ships, and `reset` clears both boards and makes a new fleet. `step` marks the chosen cell as a hit or miss and calls `check_win`, which ends the game at 2 + 3 + 4 = 9 hits. The win pays 10, and every other shot gets `reward()`, a TODO I filled in. The game never says when a ship sinks, and it lets you shoot a cell twice.[1] I only changed `reward()`. Every agent, random included, only picks untargeted cells, so a game lasts at most 36 turns.



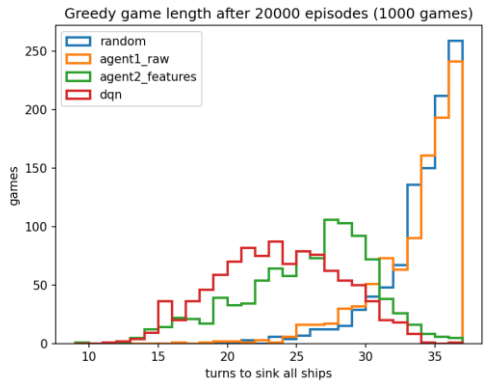


Figure 1 (left) and Figure 2 (right). Game length over 1,000 random games, and over 1,000 greedy games per trained agent.

Random games took 33.5 turns on average (median 34), and 26% went all 36 turns (Figure 1). That fits the math. The game ends on the last of the 9 ship cells, and in a random order the last of 9 cells out of 36 comes at shot 9 × 37 / 10 = 33.3 on average, and is the 36th shot with probability 9/36 = 25%. A random shooter has to clear almost the whole board.[2]

The fleet is hidden, so this is a POMDP. The board records every shot and its result, so it's a sufficient statistic for what the agent can know about the fleet, which makes it a Markov state.[3] Agent 1 uses the raw board. That's up to $3^{36} \approx 1.5 \times 10^{17}$ states, and it saw 600,122 different ones in 20,000 games. Almost every board it visited was new, with all Q-values still at 0, so its greedy policy is random tie-breaking.[3] For agent 2, I describe the cell being shot instead of the whole board, so the action is "shoot a cell of this type" and the agent shoots whichever untargeted cell has the best type. Each cell has three features: `hits` (hit neighbors: 0, 1 or 2+), `line` (whether it continues two hits in a row), and `fit` (how many

placements of a 2, 3 or 4 ship cover it without covering a miss, as min(count // 4, 4)). That's 3 × 2 × 5 = 30 states, and it visited 25 of them.[3]

The reward is +1 for a new hit and -1 for a miss, plus the 10 for the winning hit. A game of T turns returns exactly 8 + 10 - (T - 9) = 27 - T, so more reward only comes from finishing sooner. Just counting hits would be the "be careful" case, since re-shooting a hit cell would keep paying.[4] The dense ±1 tells the agent after every shot whether that kind of cell was a good choice, which I think is why the agents learned within 100 games. With $\gamma = 0.9$, a +10 twenty turns away is worth $10 \times 0.9^{20} \approx 1.2$, so the agents mostly go for hits soon, which in Battleship is nearly the same thing as finishing fast.[4] DQN uses the same reward, so the only difference between the agents is the representation.[4]

Each agent trained for 20,000 episodes with one seed. The tabular agents used $\varepsilon = 0.1$, $\gamma = 0.9$, $\alpha = 0.1$, and DQN used the starter config ($\varepsilon = 0.1$, $\gamma = 0.9$, Adam $\alpha = 0.001$, $B = 32$, $M = 100{,}000$, an update every 4 turns, $C = 500$).

Table 1. Average turns per game. Checkpoints use 500 greedy ($\varepsilon = 0$) games. "Training" is the last 1,000 training games ($\varepsilon = 0.1$). Random is 33.5.

Agent	500	5,000	10,000	20,000	Training
Agent 1, raw board	33.1	33.0	33.2	33.1	33.4
Agent 2, all features	23.1	23.4	23.0	24.9	23.7
Agent 2, no `hits`	29.1	27.0	33.2	31.2	29.5
Agent 2, no `line`	23.1	24.5	24.3	23.6	24.7
Agent 2, no `fit`	23.7	24.0	23.9	25.2	24.5
DQN	23.9	22.9	23.2	23.5	23.3
DQN, no target network	22.6	22.6	23.0	22.4	23.2

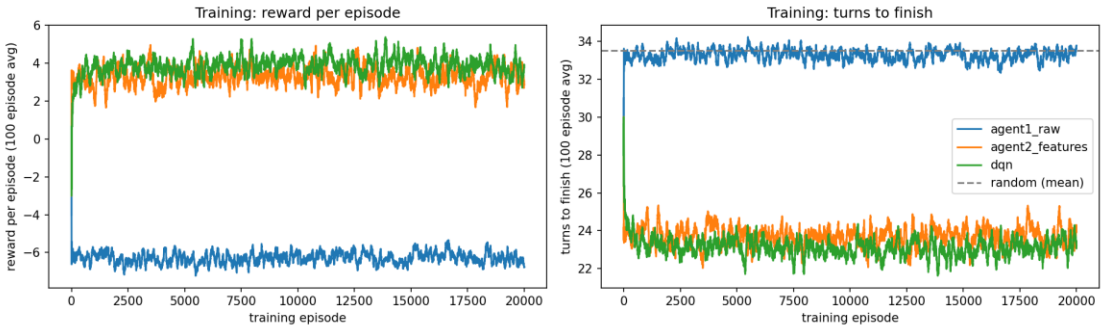


Figure 3. Training reward and turns, 100-episode moving averages.

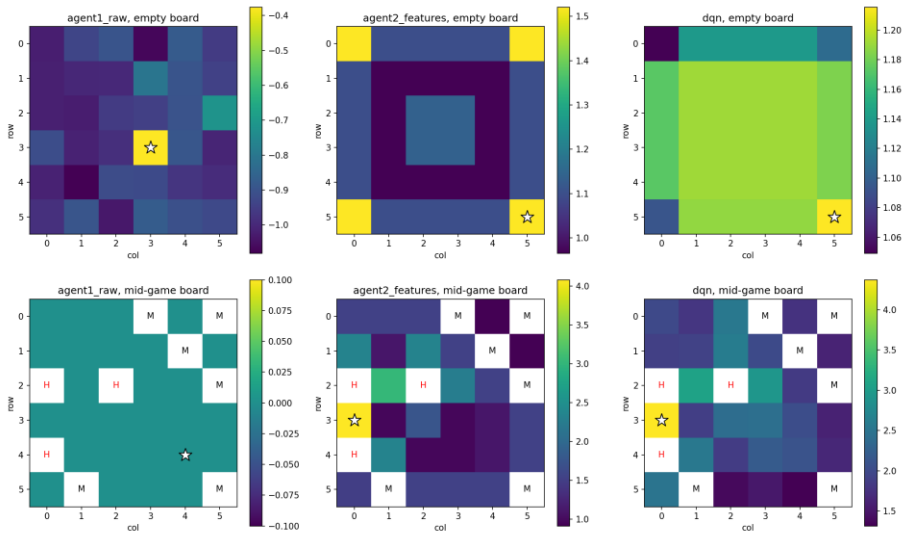
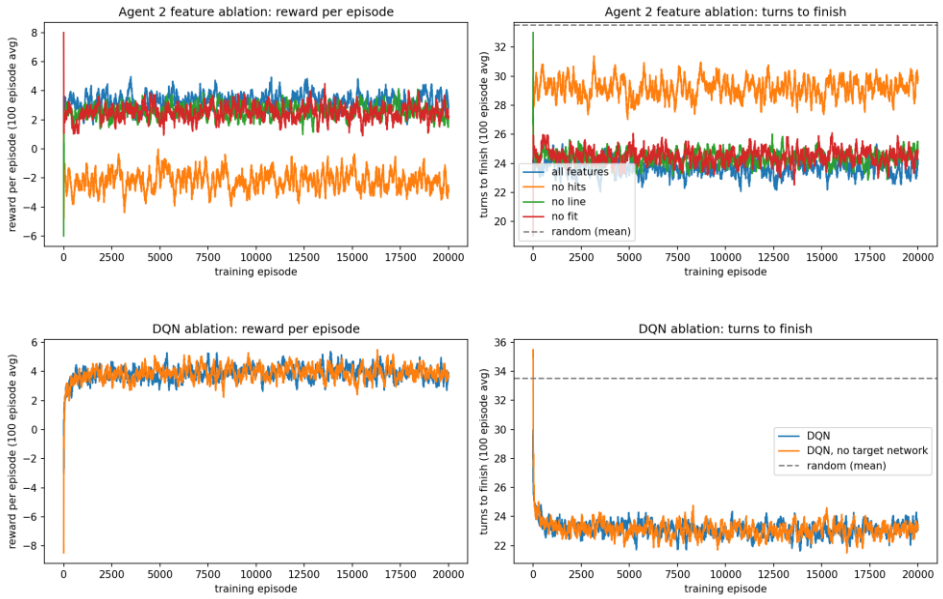


Figure 4. Q-value of each cell on an empty board (top) and a mid-game board (bottom). H = hit, M = miss. The star is the greedy pick.

On the mid-game board, agent 2 and DQN both pick (3, 0), between the two hits in column 0, and agent 1's values are all 0 (Figure 4).[5] They differ in how they hunt. DQN's empty-board values are nearly flat (1.05 to 1.21), but agent 2 opens in a corner, because among cells with no hit next to them `fit` = 1 has Q = 1.52 and `fit` = 4 (the open middle) has 1.14. That's backwards. I think low-`fit` cells mostly come up late in a game, when the +10 is closer, and since agent 2's state isn't Markov, one Q-value averages over the whole game.[5] That would also explain why its greedy games (25.3 turns) are worse than its ε = 0.1 training games (23.7), since random shots break up the bad hunting order. So both learned to hunt and then finish off a ship, they differ in the hunting, and agent 1 learned nothing (Figure 2).[5]

I didn't sweep γ or ε.[6] γ sets how much of the target comes from Q(s') instead of the immediate ±1. Lower γ means going for the most likely hit right now, which is already close to optimal, so I'd expect little change. Higher γ leans on Q(s'), which for agent 2 is the part blurred by averaging, so I'd expect it to do worse. ε controls the data the agent collects. Each of agent 2's Q-values is an average over whichever boards the behavior policy reaches, so ε changes what it learns, not just how many shots it wastes. For DQN, ε decides what ends up in the replay buffer.[6] I'd test this by changing one at a time (γ in {0.5, 0.9, 0.99}, ε in {0.01, 0.1, 0.3}) and comparing greedy turns at the same checkpoints.[6]

Agent 2 and DQN both beat random by 8 to 10 turns, and agent 1 didn't beat it at all (Table 1).[7] Agent 2 learned fastest. It averaged 23.7 turns in its first 100 games and its last 1,000, since 30 table entries each get thousands of updates in 100 games. DQN went from 24.2 over its first 500 games to 23.1 after episode 1,000, and it won the final greedy games 23.2 to 25.3. So the order flips later, but they were tied at 10,000 episodes (23.0 vs 23.2). Agent 2's top three Q-values are within 0.05 of each other, so with α = 0.1 the update noise keeps reordering them.[7]



Figures 5 and 6. Agent 2 with one feature removed (top), and DQN with and without a target network (bottom).

`hits` is the feature that matters. Without it, agent 2 is close to random (29.5 turns in training).[8] Cell types with no hit next to them have Q between 0.9 and 1.5, and the 19 highest types all have one, so `hits` is what switches the agent from hunting to finishing off a ship. Dropping `line` or `fit` costs about a turn. `line` only helps once a ship has two hits, and `fit` was supposed to help with hunting but its values are backwards.[8] No ablation changed sample efficiency, since every version flattened out within 100 games (Figure 5). Removing DQN's target network didn't hurt either (22.4 vs 23.5 greedy, Figure 6).[8] A target network keeps the bootstrap target from moving with each update. But games here are at most 36 turns, $\gamma = 0.9$ shrinks the Q(s') term, and most of each target is the immediate ±1, so errors in Q(s') don't build up. I'd expect it to matter with γ near 1 or longer games.[8] Overall, good features got most of what DQN did on a 6 × 6 board (8.2 of its 10.3 turns saved). DQN's edge is that I didn't have to design anything, and its state is Markov.

For question 9, I asked a separate Claude agent, which hadn't seen my analysis, why DQN outperformed agent 2 (prompt and answer in the appendix).[9] Its main claim was that most of the 2-turn final gap isn't a better representation, since agent 2 tied or beat DQN at the first three checkpoints and then its greedy play fell below its $\varepsilon = 0.1$ play. That's right, and its numbers check out. It also pointed out something I'd missed, which I added to question 6: an aliased Q-value is weighted by how often the behavior policy reaches each board.[9] It left out that agent 2's error isn't just random reordering, since `fit` is backwards in one consistent direction (the corner opener in Figure 4). Its claim that the CNN wins by telling apart two hits capped by misses from two open ones is unsupported, since nothing I ran looks at what the CNN represents.[9] What does support a small real DQN edge is the training curves (23.1 vs 23.7 turns after episode 1,000) and fewer long games (8.4% vs 17.1% took 30 or more turns). With one seed, I can't tell if that half turn would hold up.[9]

Appendix: Generative AI use

Every prompt is listed below.

Tool: Claude (Anthropic) through Claude Code.

Prompts, in order, verbatim

1.

> i want you to use markitdown on lab4 rl learning representations.pdf. also i added two pdfs for my final lab submisisons for lab 2 and 3 to have copies off. for future lab reports i got a really good grade on lab 2 so it is good to note. maybe for like agent.md or something. also commit everytjhing

2.

> make sure to use starter file. make code as simple as possible and use the md inside of the lab4 folder as ground truth of what needs to be done.Act as an expert Reinforcement Learning engineer but college student and also write like me and make realistic and help me complete my Python code for Lab 4: Representation Learning via Modern RL. The objective is to evaluate when Deep Q-Networks (DQN) provide meaningful advantages over classical Q-learning with hand-engineered features. We are training agents to play a simplified, partially observable version of Battleship. The environment rules are: Single-player version where only the agent selects targets on a hidden board. The fleet contains exactly three ships with lengths of 2, 3, and 4. The board is a 6x6 grid. Using the starter code provided below, please generate simple, clean, and well-commented Python code to fulfill the following lab requirements:1. Random Agent BaselineSimulate 1,000 games with randomly generated actions. Plot and save the distribution over game lengths. 2. Classical Q-Learning ImplementationImplement a classical Q-learning agent with its respective Q-values, policy, and learning update rule. Define an effective reward function, $r(s', a, s)$, to incentivize the agent to win quickly. Train Agent 1 using the raw grid board as the state (recording hits, misses, and untargeted cells). Train Agent 2 using a creatively hand-engineered state representation of your own design that aligns with the agent's action granularity. 3. Deep Q-Network (DQN) ImplementationFill in the TODO sections of the provided Agent class. Implement the replay buffer, target network updates (update_Q_target), forward pass (forward_pass), action selection (pi), batch creation (make_batch), and network optimization step (update_Q_value). Use the current board as the state for this agent and define an appropriate reward function. 4. Training & CheckpointingWrite a training loop for all three agents. Record the average

reward and average turns to completion during training. Save the models at specific checkpoints: 500; 5,000; 10,000; and 20,000 training episodes. Generate visualizations of the learned policies for each agent (e.g., target-selection heatmaps, action distributions, or estimated value functions). 5. Ablation StudiesFor the engineered representation in classical Q-learning (Agent 2), remove one feature at a time and measure the effect on sample efficiency and final performance. For the DQN agent, remove or modify one major component (such as the target network, replay buffer, or a specific CNN layer) and measure the effect on sample efficiency and final performance. Keep the code as simple as possible. Please output the code necessary to complete these tasks based on this starter code:

Claude then asked three multiple choice questions. My answers: only let agents shoot cells that haven't been shot yet (mask targeted cells), the random baseline picks a random untargeted cell, and the DQN ablation removes the target network.

3.

is it still running?

4.

is it still running?

5.

ok now using my writing style and last reports, draft the report for me. look at md doc and make sure everything is covered.

6.

do q9 for me and keep going till done

For question 9, Claude Code sent this prompt to a separate Claude agent that hadn't seen my analysis or report draft:

I'm a student in a reinforcement learning class. For my lab report I need to ask an AI assistant to analyze one of my experiments, and then I'll critically evaluate its explanation. Please give me your own analysis.

The experiment is in `C:\Users\grudzinskio\School Repos\RL-Labs\lab4_learning_representations\`. Use only `Lab_4_Oliver_Grudzinski.ipynb` (the code and its outputs) and the CSVs in `results/` (`checkpoint_eval.csv`, `training_curves.csv`, `final_games.csv`, `ablation_summary.csv`, `ablation_curves.csv`, `random_games.csv`). Do NOT open `report_draft.md`, `prompts_appendix.md`, or anything in other folders. Don't modify any files.

> Setup in short: single-player Battleship on a 6x6 board with ships of length 2, 3 and 4. Agents can only shoot untargeted cells. Reward is +1 per new hit, -1 per miss, and +10 for the winning shot. There are three agents, each trained for 20,000 episodes. Agent 1 is tabular Q-learning on the raw board. Agent 2 is tabular Q-learning on hand-engineered per-cell features. The third is a DQN with a CNN over the board.
>
> Question: why did the DQN agent outperform agent 2 (the hand-engineered tabular agent)? Explain the mechanism you think is responsible and back it up with numbers from the results.
>
> Reply with your explanation as you'd give it to me, around 250 to 400 words. That's the text I'll be quoting and evaluating, so write it as the answer itself, not as notes about what you did.

Its answer, verbatim:

> Most of the gap isn't the CNN learning a better representation. It comes from agent 2's final greedy policy ending up in a bad spot. Under that there's a smaller real gap, and I'd blame state aliasing for it.
>
> The headline numbers are 23.24 turns for DQN and 25.30 for agent 2 over 1000 greedy games, a difference of 2.07 (SE about 0.2). The checkpoints tell a different story. Agent 2's greedy evals were 23.09, 23.40 and 22.98 at 500, 5000 and 10000 episodes, so it beat or tied DQN (23.88, 22.88, 23.16). Then it jumped to 24.91 at 20000. Meanwhile agent 2's training turns, which include epsilon = 0.1 random shots, stayed flat at about 23.7 from episode 1000 to 20000. So the final agent 2 plays worse greedily (25.3) than it did while exploring (23.7). DQN doesn't do this: it trains at about 23.0 to 23.3 and plays greedily at 23.24.
>
> Here's the mechanism. Agent 2 maps every cell to one of 30 feature types (it saw 25), so lots of very different boards share one Q-table entry. Each Q(type) becomes an average over the true states behind it, weighted by how often the epsilon-greedy behavior policy produces them. With a constant alpha = 0.1, those averages keep drifting with recent games. Several types have close values, so their ranking keeps flipping. Greedy play commits completely to whatever ranking the table had at episode 20000, and that one was bad. Random exploration hid the problem during training.
>
> The real advantage is smaller and shows up in the training curves, where both agents use the same epsilon. After episode 1000, DQN averages 23.0 to 23.3 turns and agent 2 averages 23.6 to 23.7, so DQN is about half a turn faster. Agent 2 hit its ceiling by episode 500, since 25 states don't leave much to learn. DQN started worse (24.15 over its first 500 episodes) and kept improving. The CNN sees the actual 5x5 pattern of hits and misses around each cell, so it can separate situations that agent 2's three features lump together, like a pair of hits already capped by misses versus one that's still open. DQN also has fewer long games: 8.4% took 30+ turns, versus 17.1% for agent 2.

Caveats: each agent got one seed, and the checkpoint evals are 500 games each (SE about 0.2 turns), so the small differences at the early checkpoints are noise. The jump from 22.98 to 24.91 is about 6 SE, so it isn't.

7.

give me just prompts for agent for question 9

8.

do i need to provide its answer

9.

commit everything and push